

FICHA 2 – INTERATIVIDADE 3D

OBJETIVOS:

- Conhecer e manipular a hierarquia de objetos de uma cena Three.js
- Selecionar objetos interactivamente numa cena Three.js



ACONSELHA-SE QUE A RESOLUÇÃO DE TODOS OS EXERCÍCIOS PROPOSTOS SEJA FEITA INDIVIDUALMENTE.

Parte I. NAVEGAR E MANIPULAR A HIERARQUIA DA CENA THREE.JS

Tal como numa qualquer ferramenta de modelação tridimensional, podemos interpretar uma cena em *Three.js* como uma hierarquia de objetos (luzes, malhas, câmara, etc.). Esta hierarquia começa no objeto **THREE.Scene** que for passado ao renderizador.

Comece por criar uma pasta para a Ficha 4. Esta deve incluir as pastas fornecidas "parte 1", "parte 2" e "ficheiro *glTF*". De seguida, mova a nova pasta para a raiz da pasta "ThreeJS Projects", utilizada na ficha anterior.

1. CENA INICIAL

Na pasta "parte 1" do material fornecido, comece por criar um ficheiro **main.js** com:

- Uma cena *Three.js*
- Um canvas 800x600
- Uma câmara na posição inicial (4, 3, 2)
- *OrbitControls*
- Uma cena fornecida (em glTF)
- Um ponto de luz branca (PointLight), na posição (3, 4, 0) e com uma intensidade de 30

Sugestão: Consulte a ficha anterior para realizar estas tarefas.

2. USAR CANVAS DEFINIDO EM HTML

Até agora foi usado o método `document.body.appendChild()` para criar o elemento `canvas` usado pelo `renderer`. Mas o documento HTML fornecido já inclui um elemento `canvas`. Para o aproveitar é necessário:

- Obter uma referência para o objeto `canvas` usando o método `getElementById()`
- Nos parâmetros do construtor do `THREE.WebGLRenderer` adicionar a expressão `{ canvas: obj }` onde `obj` é a referência para o objeto `canvas` obtida no ponto anterior
- Eliminar a chamada a `appendChild()`

Isto é:

```
let meuCanvas = document.getElementById( 'meuCanvas' )
let renderer = new THREE.WebGLRenderer( { canvas: meuCanvas } )
```

3. NAVEGAR HIERARQUIA DE OBJETOS

Utilize a consola JavaScript do seu cliente Web para analisar a hierarquia de objetos. Para isso, envie uma mensagem para a consola (`console.log`) com a variável que guarda a referência do objeto `THREE.Scene`, criado no ponto anterior. Em particular veja os valores das propriedades: `parent`, `children`, `visible`, `geometry`, `castShadow`, e `receiveShadow` e `material` dos objetos do tipo `THREE.Mesh`. No material, repare também nas propriedades `transparent` e `opacity`.

Na variável que armazena a referência do objeto `THREE.Scene`, altere dinamicamente na consola do browser a propriedade da visibilidade de um dos objetos.

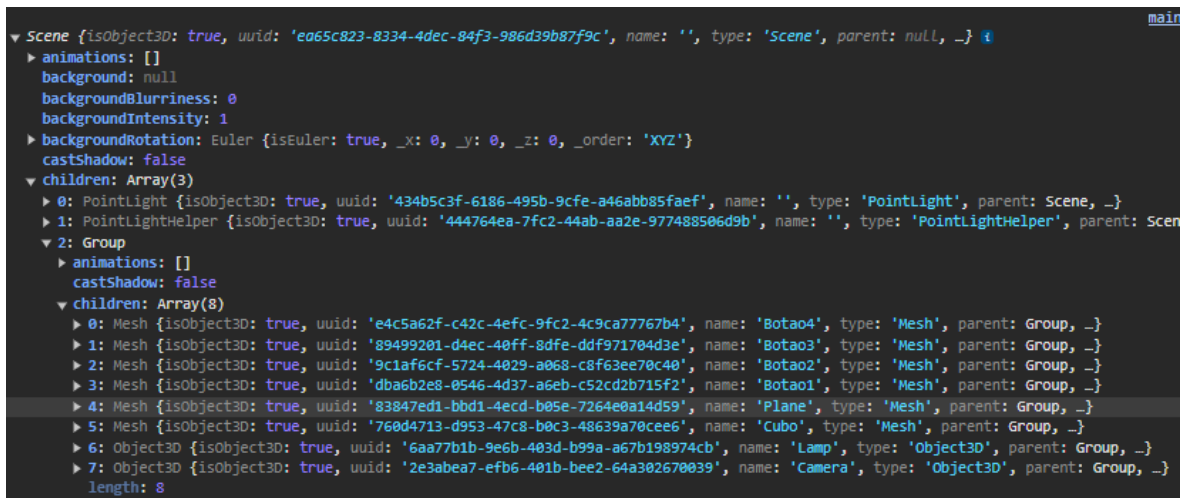


FIGURA 1 – HIERÁRQUIA DA CENA NA CONSOLA

Alternativamente, em vez de enviar uma mensagem para a consola, pode expor variáveis do módulo da seguinte forma:

```
window.nomeVariavelNaJanela = nomeDaVariavelNoModulo
```

Exemplo: `window.cena = cena`

Para imprimir uma variável exposta na consola basta escrever o nome desta.

Adicionalmente, pode invocar métodos sobre as variáveis expostas. Expondo a cena criada, por exemplo, pode aceder ao objeto “Cubo” na consola com o comando: `cena.getObjectByName(“Cubo”)`, sem ter que explorar a restante hierarquia.

```
> cena.getObjectByName('Cubo')
< Mesh {isObject3D: true, uuid: '6bf97e45-0eb1-4fa2-8803-9123229b2010', name: 'Cubo', type: 'Mesh', parent: Group, ...}
  > animations: []
  > castShadow: false
  > children: []
```

FIGURA 2 – FUNÇÃO GET OBJECT BY NAME

4. SOMBRAS, PERCORRER TODOS OS ELEMENTOS DA HIERARQUIA

Como pode reparar, por omissão, o *Three.js* não projeta sombras. Para alterar este comportamento deve colocar a **true** as seguintes propriedades:

- **castShadow** dos pontos de iluminação que podem fazer sombra (i.e., **THREE.PointLight**)
- **shadowMap.enabled** do objeto **THREE.WebGLRenderer**
- **castShadow** dos objetos **THREE.Mesh**
- **receiveShadow** dos objetos **THREE.Mesh** sobre os quais serão projetadas sombras

Altere estas propriedades para que os cubos projetem sombras sobre o plano.

Sugestão 1: O método **traverse()** permite percorrer todos os elementos desta hierarquia e invocar uma *callback* para cada um deles. Este método percorre (recursivamente ou não) o vetor **children** de cada nó da hierarquia.

Assim, para efetuar alguma manipulação em todos os elementos de uma cena *glTF* importada devemos usar:

```
cena.traverse( function (elemento) {
    // fazer alguma coisa com elemento
});
```

Sugestão 2: A propriedade **isMesh** é **true** nos objetos **THREE.Mesh**.

Sugestão 3: A cena só conterá objetos **THREE.Mesh** depois de ser carregado o ficheiro *glTF*. **Como tal, estas operações sobre esses objetos devem ser efetuadas dentro do callback do carregador.**

5. TRATAR INTERAÇÃO COM BOTÕES HTML

Adicione código JavaScript para que cada um dos botões HTML façam as alterações correspondentes sobre o cubo grande.

Sugestão 1: Na interação com os botões, utilize o construtor **THREE.Color()** para criar uma cor e, se pretender aplicar um material mais polido e brilhante ao objeto use, por exemplo, o construtor:

```
let material_novo = new THREE.MeshStandardMaterial( {
    metalness: 1,    // entre 0 e 1
    roughness: 0.5 // entre 0 e 1
} )
```

Sugestão 2: Utilize variáveis globais para guardar estados anteriores e facilitar o acesso a objetos.

Parte II. INTERAÇÃO COM OBJETOS 3D: RAYCASTER

Para selecionar objetos numa vista 3D é preciso converter o ponto 2D do cursor de rato no ecrã numa reta em 3D (em direção à cena) e identificar todos os objetos 3D cuja geometria é atravessada por esta reta. Esta operação é feita por um objeto **THREE.Raycaster** com ajuda de um vetor 2D que irá receber as coordenadas do rato:

```
let raycaster = new THREE.Raycaster()
let rato = new THREE.Vector2()
```

A variável **rato** deverá ser atualizada quando for necessário (por exemplo, quando existir movimento do rato ou quando acontecer um clique):

```
window.onclick = function(evento) {
    rato.x = (evento.clientX / Largura Canvas) * 2 - 1
    rato.y = -(evento.clientY / Altura Canvas) * 2 + 1
    // invocar raycaster
}
```

Note-se que as coordenadas de **rato** estão ao intervalo $[-1, 1]$ enquanto **evento.clientX** e **evento.clientY** estão nos intervalos $[0, \text{Largura Canvas}]$ e $[0, \text{Altura Canvas}]$ respetivamente e que, no canto superior esquerdo, as coordenadas de rato são $(-1, 1)$ enquanto **clientX** e **clientY** são zero.

Nota: Para ter um *canvas* com toda a área disponível do cliente (browser), usar como valores de largura e altura: **window.innerWidth** e **window.innerHeight**.

Para invocar o *raycaster* podemos criar uma função com a seguinte estrutura:

```
function pegarTodos() {
    raycaster.setFromCamera(rato, camara)

    let intersetados = raycaster.intersectObjects(candidatos)

    intersetados.forEach( function (i) {
        // fazer o que houver a fazer com cada intersetado
        . . . i.object . . .
    });
}
```

Onde **candidatos** é um vetor que contém todos os objetos selecionáveis usando o *raycaster*.

Ou, se só nos interessar apenas o primeiro objeto intersetado:

```
function pegarPrimeiro() {  
    raycaster.setFromCamera(rato, camara)  
  
    let intersetados = raycaster.intersectObjects(candidatos)  
  
    if (intersetados.length > 0) {  
        // fazer o que houver a fazer com o primeiro intersetado  
        . . . intersetados[0].object . . .  
    }  
}
```

1. CENA INICIAL

Começando com a pasta “parte 2” do material fornecido, crie um ficheiro **main.js** com:

- Uma cena *Three.js*
- Uma câmara na posição inicial (4, 3, 2)
- Um canvas `window.innerWidth` x `window.innerHeight`
- *OrbitControls*
- A cena fornecida (em glTF).
- Um ponto de luz branca, na posição (5, 6, 0) e com uma intensidade de 30

Nota: a cena 3D é exatamente igual à da parte anterior e pode aproveitar o ficheiro **main.js** que guardou a meio da parte anterior.

2. TRATAR INTERAÇÃO COM OBJETOS 3D

Adicione o código *JavaScript* necessário para que ao clicar num dos cubos pequenos, o cubo grande fique com o mesmo material.

Sugestão: utilize as seguintes expressões para construir um vetor de candidatos:

```
let botoes = []           // inicializa vetor vazio  
botoes.push(x)           // adiciona o elemento x ao vetor botoes  
nome.includes('Texto')   // devolve verdadeiro se 'Texto' existir na string nome
```